

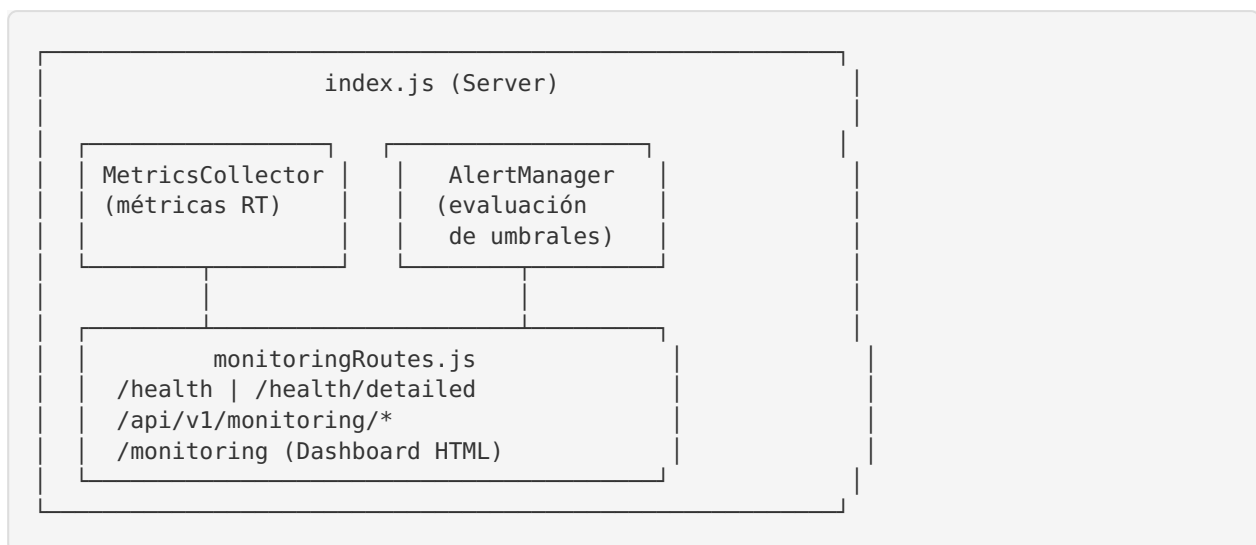


Horizon Medical — Sistema de Monitoreo

Descripción General

El sistema de monitoreo de Horizon Medical WSS proporciona visibilidad en tiempo real del estado del servidor, conexiones WebSocket, calidad de señales ECG y alertas de eventos críticos. Está diseñado para entornos médicos donde la disponibilidad y calidad del servicio son esenciales.

Arquitectura



Endpoints

Health Checks

GET /health — Health Check Simple

Diseñado para load balancers y checks de uptime.

Respuesta (200 o 503):

```
{
  "status": "healthy",
  "timestamp": "2026-05-15T17:00:00.000Z",
  "uptime": 3600
}
```

GET /health/detailed — Health Check Detallado

Incluye métricas completas, alertas activas, estado de componentes y sesiones ECG.

Respuesta (200 o 503):

```
{
  "status": "healthy|degraded|critical",
  "timestamp": "...",
  "version": "2.1.0",
  "components": {
    "mongodb": { "status": "healthy", "readyState": 1, "readyStateText": "connected" }
  },
  "websocket": { "status": "healthy", "connections": 5, "peakConnections": 12 },
  "ecgPipeline": { "status": "healthy", "packetLossRate": 0.5, "totalPackets":
1000, "activeDevices": 2 }
},
  "metrics": {
    "uptime": 3600,
    "connections": { "current": 5, "total": 100, "rejected": 2, "peak": 12, "byRole":
{ "device": 3, "monitor": 2 } },
    "messages": { "received": 5000, "processed": 4998, "errors": 2 },
    "latency": { "min": 0.1, "max": 15.5, "avg": 1.2, "p95": 5.0, "p99": 12.0 },
    "system": { "cpuUsage": 15.5, "memoryUsage": 45.2, "heapUsed": 52428800, "eventLoopLag": 2.5 }
  },
  "alerts": { "active": [], "count": 0, "criticalCount": 0 },
  "sessions": { "active": 2, "details": [...] },
  "server": { "nodeVersion": "v20.x", "platform": "linux", "hostname": "horizon-wss",
"cpus": 4 }
}
```

API de Monitoreo

GET /api/v1/monitoring/metrics

Snapshot completo de todas las métricas actuales.

GET /api/v1/monitoring/history?count=60

Serie temporal de métricas (por defecto, últimos 60 snapshots = ~10 minutos).

Parámetros:

- **count** (int): Número de snapshots a retornar (default: 60, max: 360)

GET /api/v1/monitoring/alerts?history=50

Alertas activas, historial y umbrales configurados.

Parámetros:

- **history** (int): Número de entradas del historial (default: 50)

GET /api/v1/monitoring/ecg-quality

Métricas detalladas de calidad de señal ECG por dispositivo.

Dashboard

GET /monitoring

Dashboard HTML interactivo con auto-refresh cada 5 segundos. Muestra:

- KPIs principales (conexiones, mensajes, latencia, sistema, calidad ECG, uptime)
- Gráficas de series temporales (conexiones/mensajes, CPU/memoria/event-loop)
- Panel de alertas activas e historial
- Tabla de dispositivos ECG con métricas por canal

Métricas Recolectadas

Conexiones

Métrica	Descripción
<code>connections.current</code>	Conexiones WebSocket activas
<code>connections.total</code>	Total acumulado de conexiones
<code>connections.rejected</code>	Conexiones rechazadas (auth, rate-limit, ban)
<code>connections.peak</code>	Pico máximo de conexiones concurrentes
<code>connections.byRole</code>	Desglose por rol (device, monitor, admin)

Mensajes

Métrica	Descripción
<code>messages.received</code>	Total de mensajes recibidos
<code>messages.processed</code>	Mensajes validados y procesados
<code>messages.errors</code>	Mensajes con errores de validación
<code>messages.rateLimited</code>	Mensajes rechazados por rate limiting
<code>messages.byType</code>	Desglose por tipo (ecg_data, device_status, etc.)
<code>messages.bytesReceived</code>	Bytes totales recibidos

Latencia

Métrica	Descripción
<code>latency.min</code>	Latencia mínima de procesamiento (ms)
<code>latency.max</code>	Latencia máxima de procesamiento (ms)
<code>latency.avg</code>	Latencia promedio (ms)
<code>latency.p95</code>	Percentil 95 de latencia (ms)
<code>latency.p99</code>	Percentil 99 de latencia (ms)

Calidad ECG

Métrica	Descripción
<code>ecgQuality.totalPackets</code>	Total de paquetes ECG procesados
<code>ecgQuality.droppedPackets</code>	Paquetes perdidos (detectados por gaps en secuencia)
<code>ecgQuality.outOfOrderPackets</code>	Paquetes recibidos fuera de orden
<code>ecgQuality.duplicatePackets</code>	Paquetes duplicados
<code>ecgQuality.packetLossRate</code>	Tasa de pérdida de paquetes (%)
<code>ecgQuality.avgSamplesPerPacket</code>	Promedio de muestras por paquete
<code>ecgQuality.devices[].snrEstimateDb</code>	Estimación de SNR por dispositivo/canal (dB)

Sistema

Métrica	Descripción
<code>system.cpuUsage</code>	Uso de CPU del proceso (%)
<code>system.memoryUsage</code>	Uso de memoria RSS vs total del sistema (%)
<code>system.heapUsed</code>	Memoria heap utilizada (bytes)
<code>system.heapTotal</code>	Memoria heap total (bytes)
<code>system.eventLoopLag</code>	Lag del event loop de Node.js (ms)
<code>system.loadAverage</code>	Load average del sistema [1m, 5m, 15m]

Sistema de Alertas

Reglas Configuradas

ID de Regla	Nivel	Umbral por Defecto	Variable de Entorno
HIGH_CONNECTIONS	WARNING	≥100 conexiones	ALERT_MAX_CONNECTIONS
HIGH_ERROR_RATE	CRITICAL	≥50 errores/min	ALERT_ERROR_RATE
HIGH_MEMORY	WARNING	≥85% memoria	ALERT_MEMORY_PERCENT
HIGH_EVENT_LOOP_LAG	CRITICAL	≥500ms	ALERT_EVENT_LOOP_LAG_MS
MON-GODB_DISCONNECTED	CRITICAL	Estado ≠ 1	—
HIGH_ECG_PACKET_LOSS	WARNING	≥5% pérdida	ALERT_ECG_PACKET_LOSS
LOW_ECG_SNR_*	WARNING	<10 dB SNR	ALERT_ECG_MIN_SNR_DB
AUTH_FAILURE_SPIKE	CRITICAL	≥10 fallos/min	ALERT_AUTH_FAILURES
HIGH_HEAP_USAGE	WARNING	≥90% heap	ALERT_HEAP_PERCENT

Niveles de Alerta

- **INFO:** Eventos informativos
- **WARNING:** Rendimiento degradado, acercándose a límites
- **CRITICAL:** Problemas de salud del sistema que requieren atención inmediata

Cooldown

Las alertas tienen un cooldown configurable (`ALERT_COOLDOWN_MS` , default: 60s) para evitar spam de logs repetitivos.

Variables de Entorno

Variable	Default	Descripción
METRICS_HISTORY_SIZE	360	Snapshots a mantener en historial
METRICS_SNAPSHOT_INTERVAL_MS	10000	Intervalo entre snapshots (ms)
ALERT_MAX_CONNECTIONS	100	Umbral de conexiones para alerta
ALERT_ERROR_RATE	50	Errores/min para alerta crítica
ALERT_MEMORY_PERCENT	85	% de memoria para alerta
ALERT_EVENT_LOOP_LAG_MS	500	Lag del event loop para alerta (ms)
ALERT_ECG_PACKET_LOSS	5	% de pérdida ECG para alerta
ALERT_ECG_MIN_SNR_DB	10	SNR mínimo (dB) antes de alerta
ALERT_AUTH_FAILURES	10	Fallos de auth/min para alerta
ALERT_HEAP_PERCENT	90	% heap para alerta
ALERT_COOLDOWN_MS	60000	Cooldown entre alertas repetidas (ms)

Archivos del Sistema

```
src/
├── services/
│   ├── metricsCollector.js    # Recolección de métricas en tiempo real
│   └── alertManager.js        # Evaluación de alertas por umbrales
├── routes/
│   └── monitoringRoutes.js    # Endpoints HTTP + dashboard HTML
├── tests/
│   └── monitoring.test.js      # 45 tests unitarios
├── docs/
│   └── MONITORING.md          # Esta documentación
```

Estimación de SNR (Signal-to-Noise Ratio)

- El sistema estima el SNR de señales ECG usando un filtro de media móvil simple:
1. Aplica filtro paso-bajo (ventana de 3 muestras)
 2. Calcula potencia de señal (varianza de la señal filtrada)

3. Calcula potencia de ruido (diferencia entre señal original y filtrada)
4. $SNR \text{ (dB)} = 10 \times \log_{10}(\text{potencia_señal} / \text{potencia_ruido})$

Rangos de referencia:

- **>20 dB:** Señal limpia ✓
- **10-20 dB:** Señal aceptable ⚠
- **<10 dB:** Señal ruidosa (posible problema con electrodos) ✗